

① Code

```
int arr[] = {1, 5, 2};
```

```
int n = arr.length;
```

```
int sum = 0;
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    sum = sum + arr[i];
```

```
}
```

Frequency

1

1

1

$n+1$

n

$2n+4$

arr →

0	1	2
1	5	2

n

3

Sum

8

i

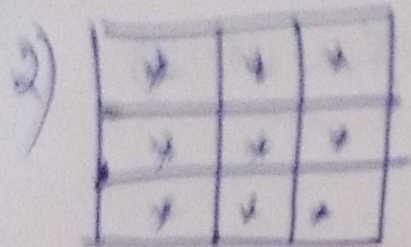
3

Rules

- ① Ignore the Constant
- ② Retain highest order term

∴ Time Complexity is

$O(n)$



Code

```
for (int i=0; i<n; i++)
{
```

```
    for (int j=0; j<n; j++)
    {
```

```
        System.out.print("*");
    }
```

```
    System.out.println();
}
```

frequency

$n+1$

$n * (n+1)$

$n * n$

n

$$n+1 + n^2 + n + n^2 + n$$

$$2n^2 + 3n + 1$$

$$n^2 + n$$

Higher order Term is n^2

So, Time complexity is $\boxed{O(n^2)}$

Matrix Multiplication Code

```
for (int i=0; i<n; i++)
{
```

```
    for (int j=0; j<n; j++)
    {
```

```
        c[i][j] = 0;
```

```
        for (int k=0; k<n; k++)
        {
```

```
            c[i][j] = c[i][j] +
                a[i][k] * b[k][j];
```

```
        }
```

```
    }
```

```
}
```

frequency

$n+1$

$n * (n+1)$

$n * n$

$n * n * (n+1)$

$n * n * n$

$$= n+1 + n^2 + n + n^2 + n^3 + n^2 + n^3$$

$$= 2n^3 + 2n^2 + n + 1$$

Time Complexity is $O(n^3)$

4)

	0	1	2	3	4
0	*				
1	*	*			
2	*	*	*		
3	*	*	*	*	
4	*	*	*	*	*

Code

Frequency

```
for (int i=0; i<n; i++)
{
```

$n+1$

```
for (int j=0; j<=i; j++)
{
```

$\frac{n(n+1)+n}{2}$

```
System.out.print("*");
}
```

$\frac{n(n+1)}{2}$

```
System.out.println();
}
```

n

Simplify we get
Time complexity is $= O(n^2)$

Calculation

$$\frac{n+1}{1} + \frac{n^2}{2} + \frac{n}{2} + \frac{n}{1} + \frac{n^2}{2} + \frac{n}{2} + \frac{n}{1}$$

$$= \frac{2n+2+n^2+n+2n+n^2+n+2n}{2}$$

$$= \frac{2n^2 + 8n + 2}{2}$$

$$= 2 \left[\frac{n^2 + 4n + 1}{2} \right]$$

$$= n^2 + \cancel{4n} + \cancel{1}$$

$$= n^2 + n$$

Higher order term is $\boxed{n^2}$
